



Vision Robot Arm

Dokumentacja projektu oraz wstępne założenia pracy inżynierskiej: śledzenie ruchu człowieka kamerą i przekazywanie ruchów kończyn górnych do ramion robotycznych.

Typ dokumentu	Opis projektu, analiza obecnego stanu i plan dalszego rozwoju
Projekt	Vision Robot Arm
Autor	[do uzupełnienia]
Promotor	[do uzupełnienia]
Uczelnia	[do uzupełnienia]
Data	17 czerwca 2026

Dokument przygotowany jako robocza baza do dalszej dokumentacji projektu i przyszłej pracy inżynierskiej. Pola oznaczone jako "do uzupełnienia" należy dostosować do wymagań uczelni.

Spis treści

1. Cel dokumentu
2. Streszczenie projektu
3. Aktualny stan implementacji
4. Architektura systemu
5. Opis głównych modułów programu
6. Planowany kierunek pracy inżynierskiej
7. Zakładany przepływ danych w wersji docelowej
8. Przewidywana struktura pracy inżynierskiej
9. Metody testowania i oceny jakości
- .0. Ryzyka techniczne i dalszy rozwój
- .1. Podsumowanie

1. Cel dokumentu

Celem dokumentu jest opisanie aktualnego projektu informatycznego oraz przygotowanie roboczego opisu tego, w jakim kierunku projekt może zostać rozwinięty w ramach pracy inżynierskiej. Dokument nie jest jeszcze pełną pracą dyplomową. Ma służyć jako materiał bazowy do rozmowy z promotorem, planowania zakresu oraz późniejszego rozbudowania rozdziałów teoretycznych, projektowych i badawczych.

Obecny kod realizuje prototyp systemu wizyjnego, który wykrywa sylwetkę człowieka na obrazie z kamery lub pliku wideo, wyznacza punkty anatomiczne, oblicza kąty stawów, rozpoznaje proste gesty i umożliwia zapis danych do pliku CSV. Docelowo system ma sterować ramionami robotycznymi tak, aby ruch ramion robota odwzorowywał ruchy wykonywane przez człowieka.

Proponowany temat roboczy: **System wizyjnego śledzenia ruchu człowieka do sterowania ramionami robotycznymi.**

2. Streszczenie projektu

Projekt Vision Robot Arm jest prototypem aplikacji komputerowej napisanej w języku Python. System wykorzystuje kamerę internetową albo nagranie wideo jako źródło obrazu. Każda klatka jest przetwarzana przez model MediaPipe Pose Landmarker, który zwraca zestaw punktów opisujących położenie głowy, tułowia, rąk i nóg. Na tej podstawie program tworzy uproszczony model sylwetki, liczy kąty wybranych stawów oraz wykrywa podstawowe gesty, na przykład uniesienie ręki albo zgięcie łokcia.

W aktualnej wersji projekt nie steruje jeszcze rzeczywistym sprzętem. Zamiast tego posiada tryb debugowania robota, w którym wybrane wartości pozy są mapowane na tekstowe komendy, takie jak ustawienie barku, łokcia lub chwytaka. Dzięki temu można sprawdzać logikę sterowania bez ryzyka uszkodzenia mechaniki.

Praca inżynierska może rozwinąć ten prototyp w kompletny system składający się z warstwy wizyjnej, warstwy interpretacji ruchu oraz warstwy sterowania ramionami robotycznymi. Najważniejszym problemem inżynierskim będzie stabilne przełożenie ruchu człowieka widocznego w kamerze na bezpieczne i powtarzalne ruchy manipulatorów.

3. Aktualny stan implementacji

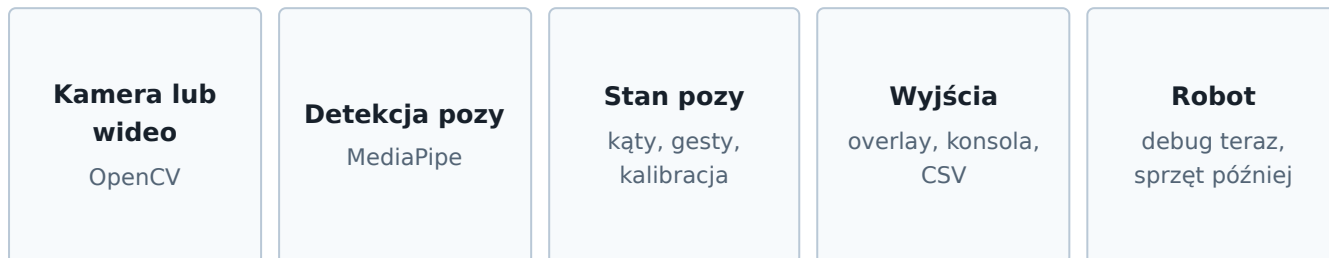
Repozytorium zawiera działający prototyp aplikacji uruchamianej z linii poleceń. Program otwiera kamerę lub plik wideo, wykonuje detekcję pozy, rysuje własny szkielet na obrazie, wypisuje dane w konsoli i opcjonalnie zapisuje przebieg do pliku CSV. Projekt zawiera również podstawowy zestaw testów jednostkowych.

Obszar	Aktualna funkcjonalność	Status
Wejście obrazu	Kamera OpenCV lub plik wideo, opcjonalne zapętlanie nagrania.	Działa w prototypie
Detekcja pozy	MediaPipe Pose Landmarker z modelem <code>pose_landmarker_lite.task</code> .	Działa w prototypie
Analiza ruchu	Obliczanie kątów łokci, barków, bioder, kolan i kostek.	Działa w prototypie
Stabilizacja danych	Proste filtrowanie dolnoprzepustowe landmarków i kątów.	Działa w prototypie
Kalibracja	Zapamiętanie neutralnej pozy i liczenie odchylenia kątów od tej pozy.	Działa, wymaga dopracowania
Gesty	Regułowe wykrywanie podniesionych rąk, rąk w bok i zgiętych łokci.	Działa w prototypie
Sterowanie robotem	Na razie tylko debugowe mapowanie pozy na tekstowe komendy.	Do rozbudowy
Rejestracja danych	Zapis timestampów, kątów, gestów i punktów ciała do CSV.	Działa w prototypie

Testy jednostkowe sprawdzają obecnie obliczanie kątów, filtrowanie, kalibrację, zapis CSV, konfigurację, gesty i mapowanie komend robota. W ostatniej weryfikacji zestaw testów zakończył się wynikiem: `Ran 8 tests - OK`.

4. Architektura systemu

Aplikacja jest podzielona na małe moduły, z których każdy odpowiada za konkretną część przetwarzania. Taki podział ułatwia dalsze rozwijanie projektu, ponieważ logika detekcji pozy, logika obliczeń i logika wyjścia nie są wymieszane w jednym pliku.



Główny przepływ działania

1. Użytkownik uruchamia program przez `python main.py`.
2. CLI tworzy konfigurację aplikacji na podstawie argumentów.
3. OpenCV otwiera kamerę lub wskazany plik wideo.
4. Każda klatka jest konwertowana do formatu RGB i przekazywana do MediaPipe.
5. MediaPipe zwraca listę punktów ciała człowieka.
6. Program wygładza dane, liczy kąty stawów i wykrywa gesty.
7. Wynik jest rysowany na obrazie i może być zapisany do pliku CSV.
8. W trybie debugowym wybrane kąty i gesty są mapowane na komendy robota.

Technologie

Technologia	Rola w projekcie
Python 3.12	Język implementacji aplikacji i testów.
OpenCV	Obsługa kamery, odczyt wideo, konwersja obrazu i wyświetlanie okna.
MediaPipe	Model detekcji pozy człowieka i zwracanie punktów anatomicznych.
NumPy	Biblioteka numeryczna używana pośrednio i przygotowana do dalszych obliczeń.
unittest	Podstawowe testy jednostkowe logiki domenowej.

5. Opis głównych modułów programu

Plik	Odpowiedzialność
<code>main.py</code>	Cienki punkt wejścia, który uruchamia funkcję CLI.

Plik	Odpowiedzialność
<code>vision_robot_arm/cli.py</code>	Definicja argumentów linii poleceń, na przykład kamera, model, smoothing i tryb robota.
<code>vision_robot_arm/config.py</code>	Konfiguracja aplikacji, wartości domyślne i walidacja podstawowych parametrów.
<code>vision_robot_arm/app.py</code>	Główna pętla programu, obsługa klatek, klawiszy, rysowania, nagrywania i kontrolera robota.
<code>vision_robot_arm/pose_tracker.py</code>	Opakowanie MediaPipe Pose Landmarker i zwracanie pierwszej wykrytej sylwetki.
<code>vision_robot_arm/state_builder.py</code>	Budowanie kompletnego stanu pozy z landmarków, kątów, kalibracji i gestów.
<code>vision_robot_arm/metrics.py</code>	Definicje kątów i funkcje obliczające kąty stawów w przestrzeni obrazu.
<code>vision_robot_arm/smoothing.py</code>	Filtry dolnoprzepustowe ograniczające drgania danych z detekcji.
<code>vision_robot_arm/gestures.py</code>	Proste reguły rozpoznawania gestów na podstawie punktów ciała i kątów.
<code>vision_robot_arm/recording.py</code>	Zapis danych pozy do plików CSV.
<code>vision_robot_arm/robot.py</code>	Abstrakcja kontrolera robota i debugowe mapowanie ruchu na komendy.

Taki podział jest dobrym punktem startowym do pracy inżynierskiej, ponieważ umożliwia stopniową wymianę warstwy debugowej na rzeczywistą komunikację ze sprzętem bez konieczności przepisywania całej aplikacji.

6. Planowany kierunek pracy inżynierskiej

Docelowy projekt ma polegać na tym, że kamera śledzi człowieka, system poprawnie identyfikuje jego kończyny, a następnie ruch kończyn górnych jest przekazywany do ramion robotycznych. Ramiona mają poruszać się w sposób zbliżony do ruchu rąk człowieka, przy zachowaniu ograniczeń mechanicznych i bezpieczeństwa.

Zakres docelowy

- Stabilne śledzenie całej sylwetki człowieka w czasie rzeczywistym.
- Wyodrębnienie najważniejszych punktów kończyn górnych: barków, łokci, nadgarstków i dłoni.
- Obliczanie kątów i przemieszczeń potrzebnych do sterowania ramionami robotycznymi.
- Kalibracja pozy neutralnej użytkownika i pozy neutralnej robota.
- Mapowanie ruchu człowieka na zakresy pracy przegubów robota.
- Filtrowanie sygnału, aby ograniczyć drgania i przypadkowe skoki pozy.
- Komunikacja z rzeczywistym kontrolerem ramienia robotycznego albo symulatorem.
- Mechanizmy bezpieczeństwa, takie jak ograniczenie prędkości, zakresu ruchu i awaryjne zatrzymanie.
- Testy dokładności śledzenia, opóźnienia oraz powtarzalności ruchu.

Przypuszczalna funkcjonalność końcowa

Po zakończeniu rozwoju system powinien umożliwiać użytkownikowi stanięcie przed kamerą, wykonanie krótkiej kalibracji, a następnie sterowanie ruchem ramion robotycznych poprzez naturalne ruchy własnych rąk. System powinien wyświetlać podgląd z kamery z naniesionym szkieletem, pokazywać aktualne kąty stawów oraz informować o stanie połączenia z robotem.

W zależności od dostępnego sprzętu możliwe są dwa warianty zakończenia projektu. Pierwszy wariant to pełne sterowanie fizycznym ramieniem robotycznym. Drugi wariant to sterowanie symulatorem lub trybem debugowym, który będzie generował komendy zgodne z przyszłym interfejsem sprzętowym. Drugi wariant nadal może być wartościowy inżyniersko, jeśli praca wykaże poprawne śledzenie, stabilne przetwarzanie i dobrze zaprojektowany interfejs komunikacyjny.

7. Zakładany przepływ danych w wersji docelowej

Wersja docelowa powinna mieć jasno wydzielony potok przetwarzania danych. Pozwoli to osobno testować warstwę wizyjną, interpretację ruchu i sterowanie robotem.

1. **Pobranie obrazu.** Kamera dostarcza kolejne klatki w czasie rzeczywistym. Ważne będą rozdzielczość, liczba klatek na sekundę i stałość oświetlenia.
2. **Detekcja punktów ciała.** Model wizyjny wykrywa punkty sylwetki człowieka. Dla sterowania ramionami najważniejsze są barki, łokcie i nadgarstki.
3. **Ocena jakości detekcji.** System powinien odrzucać punkty o zbyt niskiej widoczności i sygnalizować użytkownikowi utratę poprawnego śledzenia.
4. **Wygładzanie sygnału.** Dane z modelu wizyjnego mogą drgać. Filtracja zmniejsza niestabilność ruchu robota, ale zwiększa opóźnienie, dlatego trzeba dobrać kompromis.
5. **Kalibracja.** Pozycja użytkownika i pozycja robota muszą mieć wspólny punkt odniesienia. Kalibracja pozwala obliczać ruch względny zamiast opierać się tylko na absolutnych kątach z kamery.
6. **Mapowanie na robota.** Kąty człowieka są przeliczane na kąty przegubów robota. Należy uwzględnić inne proporcje ramion, ograniczenia mechaniczne i zakresy pracy serw.
7. **Wysłanie komend.** Komendy trafiają do kontrolera sprzętowego, na przykład przez USB, UART, sieć lokalną albo dedykowany protokół zależny od wybranego robota.
8. **Informacja zwrotna.** System powinien monitorować, czy robot przyjął komendę, czy nie przekracza limitów i czy nie wystąpił błąd komunikacji.

Przy realnym sprzęcie sterowanie musi zawierać ograniczenia bezpieczeństwa. Robot nie powinien wykonywać gwałtownych ruchów wynikających z chwilowego błędu detekcji, zasłonięcia ręki albo utraty obrazu z kamery.

8. Przewidywana struktura pracy inżynierskiej

Poniższy układ może posłużyć jako wstępna propozycja rozdziałów. Ostateczny kształt powinien zostać dopasowany do wymagań uczelni i promotora.

Rozdział 1. Wprowadzenie

Opis problemu, motywacja, cel pracy, zakres projektu oraz krótkie przedstawienie zastosowań systemów śledzenia ruchu i teleoperacji robotów.

Rozdział 2. Podstawy teoretyczne

Omówienie komputerowego widzenia, detekcji pozy człowieka, punktów anatomicznych, filtracji sygnałów oraz podstaw sterowania manipulatorami. W tym rozdziale można wyjaśnić, czym są landmarki, kąty stawów, kalibracja i ograniczenia mechaniczne robotów.

Rozdział 3. Analiza wymagań

Wymagania funkcjonalne i нефункционаłne: działanie w czasie rzeczywistym, dokładność śledzenia, obsługa kamery, zapis danych, tryb debugowy, bezpieczeństwo i możliwość dalszej integracji ze sprzętem.

Rozdział 4. Projekt systemu

Architektura aplikacji, opis modułów, przepływ danych, format zapisywanych danych oraz projekt interfejsu między aplikacją a kontrolerem robota.

Rozdział 5. Implementacja

Opis wykorzystanych bibliotek, sposobu uruchamiania aplikacji, detekcji pozy przez MediaPipe, liczenia kątów, wygładzania, kalibracji, rozpoznawania gestów i mapowania na komendy robota.

Rozdział 6. Testy i eksperymenty

Testy jednostkowe, testy z kamerą, testy z nagraniami, pomiar opóźnień, ocena stabilności kątów oraz porównanie ruchu człowieka z ruchem robota albo symulatora.

Rozdział 7. Wnioski i dalszy rozwój

Podsumowanie efektów, opis ograniczeń, ocena osiągnięcia celu oraz możliwe kierunki rozbudowy: lepsza filtracja, śledzenie 3D, obsługa dłoni, bardziej zaawansowany robot i sprzężenie zwrotne.

9. Metody testowania i oceny jakości

Aby praca inżynierska miała mocną część praktyczną, warto przewidzieć konkretne metody oceny działania systemu. Same zrzuty ekranu z działającej kamery mogą być niewystarczające. Lepiej przygotować mierzalne kryteria.

Kryterium	Proponowany sposób pomiaru
Stabilność detekcji	Pomiar odchylenia kątów, gdy użytkownik stoi nieruchomo.
Opóźnienie	Pomiar czasu od ruchu użytkownika do wygenerowania komendy robota.
Powtarzalność	Kilkukrotne wykonanie tego samego ruchu i porównanie uzyskanych zakresów kątów.
Odporność na utratę punktów	Sprawdzenie zachowania programu przy zasłonięciu ręki lub wyjściu poza kadr.
Bezpieczeństwo robota	Weryfikacja limitów prędkości, zakresów ruchu i reakcji na brak poprawnej detekcji.
Poprawność zapisu danych	Analiza plików CSV i zgodności timestampów, kątów oraz gestów.

W obecnym projekcie istnieją już testy jednostkowe dla części logiki. W dalszym rozwoju warto rozszerzyć je o testy walidacji parametrów, zachowania kalibracji, odporności na brak landmarków oraz mapowania komend robota dla różnych pozycji wejściowych.

10. Ryzyka techniczne i dalszy rozwój

Ryzyka

- Drgania punktów wykrywanych przez model wizyjny.
- Opóźnienie między ruchem człowieka a ruchem robota.
- Utrata śledzenia przy zasłonięciu ręki albo słabym oświetleniu.
- Różnice między biomechaniką człowieka a geometrią robota.
- Niebezpieczne ruchy wynikające z błędnego odczytu pozy.
- Ograniczona dokładność kątów liczonych z obrazu 2D.

Możliwe usprawnienia

- Dokładniejsza walidacja parametrów wejściowych.
- Lepsze zabezpieczenie zapisu CSV przed nadpisaniem plików.
- Kalibracja odrzucająca zbyt słabą detekcję.
- Filtry dobrane eksperymentalnie do ruchu ramion.
- Sterownik sprzętowy z limitami prędkości i zakresu ruchu.
- Tryb symulacji robota przed testami na fizycznym ramieniu.

W obecnym kodzie najbardziej naturalnym kolejnym krokiem jest dopracowanie warstwy pośredniej między stanem pozy a robotem. Powinna ona przyjmować stabilne,

przefiltrowane wartości kątów i zamieniać je na komendy zgodne z wybranym sprzętem. Dobrą praktyką będzie pozostawienie trybu debugowego oraz dodanie drugiego kontrolera, który komunikuje się z rzeczywistym urządzeniem.

11. Podsumowanie

Projekt Vision Robot Arm stanowi dobry punkt startowy do pracy inżynierskiej związanej z wizyjnym śledzeniem ruchu człowieka i sterowaniem ramionami robotycznymi. Obecna implementacja realizuje podstawowe elementy systemu: pobieranie obrazu, detekcję pozy, liczenie kątów, wygładzanie danych, kalibrację, rozpoznawanie prostych gestów, zapis CSV i debugowe mapowanie na komendy robota.

Najważniejszym etapem dalszej pracy będzie przejście od samego śledzenia pozy do bezpiecznego sterowania ruchem robota. Wymaga to dopracowania kalibracji, filtracji, mapowania kinematycznego, komunikacji ze sprzętem oraz metod testowania. Dzięki modularnej strukturze obecnego kodu te elementy można rozwijać stopniowo, bez burzenia istniejącego prototypu.

Dokument można później rozszerzyć o zrzuty ekranu działania programu, schemat stanowiska testowego, opis konkretnego modelu ramienia robotycznego oraz wyniki eksperymentów.